

PandOSsh Phase 1: Technical Documentation

1. Project Overview

PandOSsh is an educational multiprocessor operating system designed to run on the μ RISCV architecture. The core purpose of this project is to implement the foundational layers of an OS kernel, starting from data structure management up to process scheduling and device handling.

The implementation described in this document has been implemented by **Yuri Disalvatore, Leonardo Carletti, Giacomo Bruno** and **Federico Serra**.

Project Roadmap

The development of PandOSsh is divided into three distinct phases:

- **Phase 1: The Queue Managers**
 - This phase implements Level 2 of the architecture.
 - It focuses on the logical management of processes and synchronization primitives (semaphores).
 - It does not involve process execution or hardware interaction but establishes the data structures required for the subsequent layers
- **Phase 2: The Nucleus**
- **Phase 3: The Support Level**

This document specifically details the implementation and architecture of **Phase 1**.

2. Repository Structure

The repository is organized to separate generic kernel headers, architecture-specific code, and phase-specific implementations.

Directory Layout

- **headers/**: Contains global header files used across different phases.
 - `const.h`: Defines system constants.
 - `types.h`: Defines core data structures like `pcb_t` (Process Control Block), `semaphore_t` (Semaphore Descriptor), and `support_t`.
 - `listx.h`: An implementation of circular, doubly-linked lists derived from the Linux

Kernel.

- **phase1/**: Contains the source code specific to Phase 1.
 - `pcb.c`, `headers/pcb.h`: Implementation of PCB allocation, queues, and trees.
 - `asl.c`, `headers/asl.h`: Implementation of the Active Semaphore List (ASL).
 - `p1test.c`: The provided test code to verify the functionality of Phase 1 modules.
- **CMakeLists.txt**: The root configuration file for the build system.
- **config_machine.json**: Configuration file for the μ RISCV emulator.
- **klog.c**: A utility library for circular log buffering.

3. Setup

Build Instructions

The project uses CMake for compilation. Follow these steps to build the kernel:

1. **Generate Build Files:** `cmake -B build`
2. **Compile:** `cmake --build build`

This process will compile the source files (`pcb.c`, `asl.c`, `p1test.c`, `klog.c`) and link them with the μ RISCV startup objects.

Execution

To run the OS in the emulator:

1. **Run the emulator:** `uriscv`
2. Load the `config_machine.json` file under Simulator > Open Configuration
3. Click on the Power On Button, then on the Continue button.
4. **Verify Output:** The emulator will load the kernel. Successful execution is indicated by the message "**So Long and Thanks for All the Fish**" followed by "System Halted" in Terminal 0. A failure results in a "Kernel Panic".

4. Module Reference

Phase 1 implements two main modules: the Process Control Block (PCB) Manager and the Active Semaphore List (ASL) Manager.

4.1. PCB Manager (`phase1/pcb.c`)

This module manages the allocation, deallocation, organization, and hierarchy of Process Control Blocks (`pcb_t`). It maintains a pool of free PCBs (`pcbFree_h`) and supports

process queues (ordered by priority) and process trees (parent-child relationships).

Key Functions:

- **void initPcbs()**
 - Initializes the pcbFree_h list by adding all elements from the static array pcbFree_table (size MAXPROC). This is called once during system initialization .
- **pcb_t* allocPcb()**
 - Returns NULL if pcbFree_h is empty. Otherwise, it removes a PCB from the free list, initializes all fields to NULL or 0 (resetting pointers and state), assigns a new PID, and returns the pointer. This ensures no stale data persists.
- **void freePcb(pcb_t* p)**
 - Inserts the PCB pointed to by p back onto the pcbFree_h list, making it available for future allocation.
- **void insertProcQ(struct list_head* head, pcb_t* p)**
 - Inserts PCB p into the queue head based on priority (p_prio). The list is ordered by descending priority (higher integer = higher priority). If priorities are equal, p is inserted after existing elements of the same priority (FIFO for equal priority).
- **pcb_t* removeProcQ(struct list_head* head)**
 - Removes and returns the first PCB (highest priority) from the queue head. Returns NULL if the queue is empty.
- **pcb_t* outProcQ(struct list_head* head, pcb_t* p)**
 - Removes a specific PCB p from the queue head. If p is not in the queue, it returns NULL. It scans the list to confirm p's presence before removal.
- **void insertChild(pcb_t* prnt, pcb_t* p)**
 - Establishes a parent-child relationship. Sets p->p_parent = prnt and adds p to prnt's child list (p_child) via the sibling list p_sib.
- **pcb_t* removeChild(pcb_t* p)**
 - Removes the first child of PCB p. It unlinks the child from the sibling list, clears its parent pointer, and returns the child. Returns NULL if p has no children.

4.2. ASL Manager (phase1/asl.c)

This module implements the Active Semaphore List (ASL). It manages semaphores (represented by integer addresses s_key) and the processes blocked on them. The ASL is

a sorted linked list of active semaphore descriptors (`sem_d_t`). Active means at least one process is waiting on that semaphore.

Key Functions:

- **void initASL()**
 - Initializes the `sem_d_free_h` (free descriptors) and `sem_d_h` (active list). Populates `sem_d_free_h` with all static descriptors from `sem_d_table`.
- **int insertBlocked(int* semAdd, pcb_t* p)**
 - Inserts PCB `p` into the queue of the semaphore with key `semAdd`:
 1. Searches the ASL for a descriptor with key `semAdd`.
 2. If found, appends `p` to its process queue.
 3. If not found, allocates a new `sem_d_t` from `sem_d_free_h`, initializes it with `semAdd`, inserts it into the sorted ASL, and then appends `p`.
 4. Returns TRUE if allocation fails (no free descriptors), otherwise FALSE.
- **pcb_t* removeBlocked(int* semAdd)**
 - Searches the ASL for `semAdd`. If found, removes the head PCB from that semaphore's queue. If the queue becomes empty, the semaphore descriptor is removed from the ASL and returned to the free list. Returns the unblocked PCB or NULL if not found.
- **pcb_t* outBlocked(pcb_t* p)**
 - Removes a specific PCB `p` from the semaphore queue it is currently blocked on (indicated by `p->p_semAdd`). Logic handles descriptor deallocation if the queue becomes empty. Returns `p` or NULL on error.
- **pcb_t* headBlocked(int* semAdd)**
 - Returns the PCB at the head of the queue for semaphore `semAdd` without removing it. Returns NULL if the semaphore is not active or the queue is empty.

Helper Functions:

- **static inline sem_d_t* getSemdFromContainer(int* key)**
 - Extract a semaphore from its list container and returns its pointer. Relies on the `container_of` list macro.

- **static semd_t* getSemdByKey(int* key)**
 - Search for a semaphore descriptor in the ASL by its key. If found return a pointer to the semaphore, else return NULL.
- **static inline void freeSemd(sem_t *sem)**
 - Insert the semaphore sem at the head of the semdFree_h list.
- **static inline semd_t* allocSemd()**
 - Allocate a semaphore descriptor from the semdFree_h list. Return a pointer to the allocated semaphore, or NULL if the free list is empty.
- **static inline void initSemd(sem_t *sem, int *key)**
 - Initialize a semaphore descriptor values with selected key, by setting its s_key to key and initializing its process queue.